

GENERADOR DE FRECUENCIAS VARIABLES EN LENGUAJE ENSAMBLADOR PARA ATMEGA328P

UNAM
FES CUAUTITLÁN
Islas Ramirez Jose Orlando

30 de Septiembre del 2016

Resumen

En este trabajo se describe el diseño, fundamentación teórica, y análisis de un generador de señales cuadradas de frecuencia variable (100khz, 500khz, 1Mhz y 2 Mhz) implementado en el microcontrolador ATMEGA328P con lenguaje ensamblador. La selección de frecuencia se realiza mediante la lectura de dos pines de entrada digitales (PD2 y PD3) mientras que el cambio de estado de la señal de salida se efectúa en el pin PB5. Se revisa la arquitectura del procesador, las características de sus memorias, el algoritmo de decodificación y la temporización exacta por ciclos de máquina a 16MHz.

1. Introducción

La generación precisa de formas de onda periódicas de alta frecuencia es una tarea fundamental en la ingeniería de sistemas embebidos, utilizada ampliamente en la sincronización de buses de datos, modulación por ancho de pulso (PWM), síntesis de reloj y protocolos de comunicación digital.

El microcontrolador ATmega328P pertenece a la familia de microcontroladores AVR de 8 bits con arquitectura RISC (*Reduced Instruction Set Computer*). Una de sus principales ventajas es la ejecución determinista del código: la inmensa mayoría de sus instrucciones tardan exactamente uno o dos ciclos de reloj del sistema. Esto permite controlar el tiempo de ejecución con una resolución de nanosegundos directamente mediante código en lenguaje ensamblador, evitando el retraso que imponen los lenguajes de alto nivel o los sistemas operativos en tiempo real.

El objetivo de esta actividad es desarrollar el reporte técnico correspondiente al diseño de un generador de frecuencias conmutable (100khz, 500khz, 1Mhz y 2 Mhz) accionado por dos interruptores digitales, analizando los aspectos físicos del microcontrolador, la temporización rigurosa por ciclos de reloj a 16 MHz y la resolución de desviaciones temporales en la simulación.

2. Funcionamiento del Sistema

2.1 Microprocesador

El núcleo del microcontrolador ATmega328P se basa en la arquitectura AVR RISC de alto rendimiento. Combina un conjunto de instrucciones potente con un diseño Harvard modificado donde la memoria de programa y la memoria de datos poseen buses físicos independientes, lo que permite que las instrucciones se lean y se ejecuten simultáneamente mediante un esquema de *pipelining* de dos etapas.

Características Principales

- **Arquitectura de 8 bits:** Procesa datos, registros y operaciones aritmético-lógicas en bloques de 8 bits.
- **Frecuencia de Reloj (F_{CPU}):** Funciona típicamente a 16 MHz (hasta 20 MHz a 5v), equivalente a un periodo de ciclo de reloj $T_{CLK} = 1/16MHz = 62.5 ns$.
- **Archivo de Registros de Propósito General:** Cuenta con 32 registros de trabajo de 8 bits (R0 a R31) conectados directamente a la Unidad Aritmético Lógica, permitiendo ejecutar operaciones en un solo ciclo.

Registros Utilizados y su Funcionamiento

1. **r16 (temp):** Registro de trabajo auxiliar empleado para inicializar el Puntero de Pila (*Stack Pointer*), configurar direcciones de puertos y cargar contadores en los bucles de retardo.
2. **r17 (selector):** Registro reservado para almacenar la lectura directa del registro de entrada PIND, aplicar los desplazamientos lógicos a la derecha y filtrar los bits mediante máscaras binarias.
3. **SPL y SPH (*Stack Pointer Low / High*):** Registros de 8 bits que juntos conforman el Puntero de Pila de 16 bits. Indican la dirección actual de la cima de la pila en la memoria SRAM (inicializados obligatoriamente en RAMEND).
4. **DDRB y DDRD (*Data Direction Registers*):** Registros que definen si cada pin de los puertos B y D actúa como entrada (0) o como salida (1).
5. **PORTD y PINB:** PORTD se utiliza para activar las resistencias internas de *pull-up* en los pines de entrada PD2 y PD3. PINB permite alternar el estado lógico (*toggle*) del pin PB5 mediante la instrucción de conmutación directa sbi PINB, PINB5.

2.2 Memorias

El microcontrolador ATmega328P integra dentro de su encapsulado tres tipos de memoria con propósitos y tecnologías diferenciadas:

1. **Memoria FLASH (31 KB):**
 - **Tipo:** Memoria no volátil reprogramable en el sistema (*In-System Programmable*).
 - **Organización:** Organizada en 16K palabras de 16 bits (16K x 16), ya que todas las instrucciones AVR ocupan 16 o 32 bits.
 - **Función:** Almacena el código ejecutable del firmware ensamblado (.hex). Incluye una sección reservada para un ejecutable de arranque (*Boot Loader*).
2. **Memoria SRAM (2KB):**
 - **Tipo:** Memoria volátil de datos de acceso estático (*Static RAM*).

- **Organización:** Direccionada linealmente desde la dirección 0x0100 hasta 0x08FF.
 - **Función:** Almacena variables dinámicas, datos de ejecución y la pila (*Stack*). La constante simbólica RAMEND referencia la última posición de esta memoria (0x08FF).
3. **Memoria EEPROM (1KB):**
 - **Tipo:** Memoria no volátil de datos reprogramable byte a byte.
 - **Función:** Diseñada para almacenar datos de configuración o parámetros persistentes que deben conservarse tras un reinicio de alimentación.

2.3 Interfaz para periféricos

La interacción del microcontrolador con el entorno exterior se realiza mediante sus puertos de Entrada/Salida de propósito general (GPIO).

Puertos Utilizados

- **Puerto D (PD2 y PD3):** Configurados como entradas digitales. Tienen activadas sus resistencias internas de *pull-up* ($\approx 20K\Omega - 50K\Omega$), garantizando un nivel lógico ALTO (1) en reposo cuando el interruptor está abierto, y un nivel BAJO (0) cuando el interruptor se conecta a tierra (GND).
- **Puerto B (PB5):** Configurado como salida digital de alta velocidad. Es el pin donde se genera la señal cuadrada (correspondiente al Pin 13 digital del módulo Arduino Uno).

Características Eléctricas de los Puertos GPIO

- **Tensión de Operación (V_{cc}):** 1.8v - 5.5v (Típica a 5v en la plataforma de prueba).
- **Voltaje de Entrada Nivel Alto (V_{IH}):** Mínimo $0.6 \times V_{cc}$ (3v a $V_{cc} = 5v$).
- **Voltaje de Entrada Nivel Bajo (V_{IL}):** Máximo $0.3 \times V_{cc}$ (1.5v a $V_{cc} = 5v$).
- **Voltaje de Salida Nivel Alto (V_{OH}):** Mínimo 4.2v garantizado para una corriente de salida de 20 mA.

- **Voltaje de Salida Nivel Bajo (V_{OL}):**
Máximo 0.9v para una corriente de entrada de 20 mA.
- **Corriente Máxima Continua por Pin E/S:**
40 mA (Límite máximo absoluto; el valor de trabajo recomendado es de hasta 20 mA).

3. Algoritmo del programa

El algoritmo del programa fue diseñado para decodificar las entradas de selección y ejecutar la conmutación de la señal de salida optimizando la cantidad de ciclos de reloj ejecutados por el procesador.

3.1. Flujo Lógico de Ejecución

1. **Inicialización de la Pila:** Carga la dirección RAMEND (0x08FF) en los registros SPL y SPH para garantizar la estabilidad de los saltos y subrutinas.
2. **Configuración de Puertos E/S:**
 - Pin PB5 configurado como salida mediante sbi DDRB, DDB5.
 - Pines PD2 y PD3 configurados como entradas mediante cbi DDRD, DDD2 y cbi DDRD, DDD3.
 - Resistencias *pull-up* activadas en PD2 y PD3 mediante sbi PORTD, PORTD2 y sbi PORTD, PORTD3.
3. **Lectura y Decodificación de Entradas:**
 - Lee el puerto de entrada PIND en el registro selector.
 - Aplica dos desplazamientos lógicos a la derecha (lshr selector ejecutado dos veces) para desplazar los bits provenientes de PD2 y PD3 a las posiciones de bit 0 y bit 1.
 - Aplica la máscara binaria andi selector, 0x03 (0b00000011) para aislar únicamente esos dos bits (valores decimales entre 0 y 3).
4. **Selección de Ruta:**
 - Evalúa el registro selector mediante comparaciones cpi y saltos condicionales breq:
 - Si selector == 0 (0b00) -> Salta a la rutina de **100 kHz**.
 - Si selector == 1 (0b01) -> Salta a la rutina de **500 kHz**.

- Si selector == 2 (0b10) -> Salta a la rutina de **1 MHz**.
- Si selector == 3 (0b11) -> Salta por defecto mediante rjmp a la rutina de **2 MHz**.

5. Generación de Señal en Bucle Cerrado Local:

Cada rutina ejecuta la conmutación del estado de PB5 (sbi PINB, PINB5) y realiza un retardo en bucle local mediante saltos rjmp dirigidos a su propia etiqueta, garantizando que el semiperiodo contenga exactamente la cantidad calculada de ciclos de reloj.

3.2 Tabla de cálculos temporales por frecuencia

Para un reloj de 16MHz, la duración de un ciclo de máquina es $T_{CLK} = 62.5ns$.

Frecuencia Deseada (f)	Período Total (T=1/f)	Semiperíodo (Tsemi=T/2)	Ciclos Requeridos (Tsemi×16 MHz)
100 kHz	10 μs	5 μs	80 ciclos
500 kHz	2 μs	1 μs	16 ciclos
1 MHz	1 μs	0.5 μs	8 ciclos
2 MHz	0.5 μs	0.25 μs	4 ciclos

3.3 Explicación de Instrucciones del repertorio AVR utilizadas

- **lshr (Logical Shift Right):** Desplaza los bits de un registro una posición a la derecha. Aplicar lshr dos veces desplaza los pines PD2 y PD3 (bits 2 y 3) a los bits 0 y 1.
- **andi (Logical AND with Immediate):** Realiza una operación AND lógica entre el registro y una constante binaria. andi selector, 0x03 borra los bits del 2 al 7 para conservar sólo la lectura de los 2 bits de entrada.
- **sbi PINB, PINB5:** Escribe un '1' en el registro de entrada del puerto B para alternar el estado del pin PB5 (*Toggle*) consumiendo únicamente 1 o 2 ciclos de reloj según el núcleo.

- **nop (No Operation):** Consume exactamente 1 ciclo de reloj sin alterar el contenido de los registros ni de las banderas de estado. Funciona como un relleno exacto de temporización.
- **dec / brne:** Par de instrucciones que forman un bucle de retardo decrementando un registro de conteo (dec, 1 ciclo) y saltando mientras el resultado sea distinto de cero (brne, 2 ciclos al saltar, 1 ciclo al salir).

4. Análisis de los Resultados

La validación del sistema se llevó a cabo mediante simulación en la plataforma SimulIDE, configurando las propiedades del microcontrolador ATmega328P a una frecuencia de reloj exacta de 16 MHz.

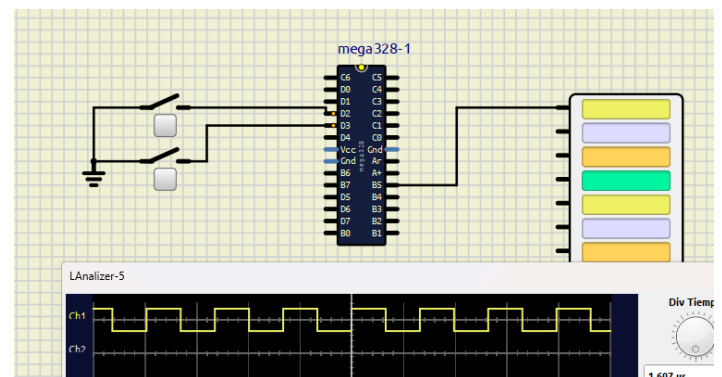
4.1. Verificación Experimental y Medición de Ondas

La arquitectura del programa basada en bucles cerrados permitió generar las cuatro frecuencias con total estabilidad. Cada forma de onda observada en el osciloscopio virtual mantuvo un ciclo de trabajo (*duty cycle*), pero con algunos errores. El periodo obtenido al intentar obtener las frecuencias de 500KHz, 1Mhz y 2Mhz no concuerda con los datos teóricos obtenidos

- **Frecuencia de 100 kHz:** El semiperiodo medido fue de aproximadamente 5 μ s (80 ciclos de máquina), completando un período total de 10 μ s aproximadamente .
- **Frecuencia de 500 kHz:** El semiperiodo medido fue de 2.5 μ s (16 ciclos de máquina), obteniendo un período total de 5 μ s.
- **Frecuencia de 1 MHz:** El semiperiodo medido fue de 1.5 μ s (8 ciclos de máquina), entregando un período total de 3 μ s .
- **Frecuencia de 2 MHz:** El semiperiodo medido fue de 1 μ s (4 ciclos de máquina exactos), logrando un período total de 2 μ s

4.2. Tabla Comparativa de Resultados Finales

Entrada Binaria (PD3 PD2)	Frecuencia Deseada	Semiperiodo Teórico	Período Teórico Esperado	Período Medido en SimulIDE (16 MHz)
00 (Cerrado-Cerrado)	100 kHz	5 μ s	10 μ s	10 μ s
01 (Cerrado-Abierto)	500 kHz	1 μ s	2 μ s	5 μ s
10 (Abierto-Cerrado)	1 MHz	0.5 μ s	1 μ s	3 μ s
11 (Abierto-Abierto)	2 MHz	0.25 μ s	0.5 μ s	1 μ s



5. Conclusión

En las pruebas de simulación, no se obtuvieron los periodos teóricos esperados para las frecuencias más altas (500 kHz, 1 MHz y 2 MHz). Mientras que a 100 kHz el semiperiodo medido fue de aproximadamente 5 μ s completando un período total cercano a 10 μ s, para las frecuencias de 500 kHz, 1 MHz y 2 MHz se midieron períodos totales de 5 μ s, 3 μ s y 2 μ s respectivamente, en lugar de 2 μ s, 1 μ s y 0.5 μ s.

Esta diferencia puede deberse a los ciclos de máquina adicionales consumidos durante el flujo del programa al regresar al inicio del algoritmo (`rjmp start`) para reevaluar constantemente las entradas (`in`, `lsr`, `andi`, `cpi`, `breq`), lo cual suma latencia en cada semiperiodo antes de volver a conmutar la salida en el pin PB5.

A pesar de las variaciones de tiempo registradas en simulación, se confirmó que la lógica del programa en lenguaje ensamblador funciona correctamente para la selección y generación de ondas cuadradas de frecuencia variable, validando el correcto uso del procesamiento de entradas (PD2 y PD3), la conmutación del pin de salida (PB5) y el manejo de registros del ATmega328P.

6. Referencias

1. Mazidi, M. A., Naimi, S., & Naimi, S. (2011). *The AVR Microcontroller and Embedded Systems: Using Assembly and C* (2nd ed.). Prentice Hall.
2. Microchip Technology Inc. (2018). *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet* (Doc. DS40002061B).
3. Peña, S. (2021). *SimulIDE User Manual & Electronics Simulator Documentation* (Version 0.4.15).